UNITAR INTERNATIONAL UNIVERSITY

FACULTY OF BUSINESS AND TECHNOLOGY

APPROVAL PAGE

Project title: SMARTPLATE: A Web-Based Intelligent System for Analyzing Restaurant Menus and Providing Personalized Nutrition and Health-Aware Meal Recommendations

Candidate: DHAARSHINI AP KARUNAKARAN

Matric No.: MC230521821

Supervisor: DR. UMAR FAROOQ KHATTAK

The undersigned certifies that the candidate has fulfilled the requirements for assessment of the Minor Project, subject to the university's applicable approval process.

Supervisor signature: _____

Date: _____

STATUS: NOT YET SIGNED — official supervisor approval required.

SMARTPLATE: A WEB-BASED INTELLIGENT SYSTEM FOR ANALYZING RESTAURANT MENUS AND PROVIDING PERSONALIZED NUTRITION AND HEALTH-AWARE MEAL RECOMMENDATIONS

DHAARSHINI AP KARUNAKARAN

MC230521821

PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE DEGREE OF BACHELOR OF INFORMATION TECHNOLOGY

UNITAR INTERNATIONAL UNIVERSITY

NOVEMBER 2026

Report status: Pre-submission draft. Candidate must review, revise, document actual evaluation and obtain required signatures before final submission.

DECLARATION

I hereby declare that the work in this project is my own except for quotations and citations that have been duly acknowledged. All software dependencies, external sources and the contribution of any coding or writing assistance must be disclosed according to the university's academic integrity requirements.

Candidate signature: _____

Date: _____ (Sign only after reviewing the complete submission.)

ABSTRACT

SMARTPLATE is a web-based prototype designed to improve access to restaurant menu information and support informed meal comparison through nutritional descriptions and explainable recommendations. The system provides a searchable directory of restaurants, detailed menu entries, per-serving nutritional values, preference-based filtering, ranked meal suggestions, and a protected administrative interface for maintaining restaurants and menu items. A deterministic rule-based recommendation algorithm first excludes unavailable or ineligible meals and then scores suitable items using calorie-target proximity, a user-selected nutritional priority, data completeness, and fiber information. The resulting ranking displays reasons and limitations for each suggestion. The implementation uses FastAPI, SQLite, Pydantic and a responsive HTML, CSS and JavaScript interface, and contains 48 illustrative menu entries across four fictional demonstration restaurants. Automated unit and API testing produced 20 passing tests during development, including validation of filtering, recommendation ranking, administration, authentication and missing-data behavior. The prototype explicitly distinguishes illustrative values from verified nutritional information and does not claim medical or food-allergy safety. Real-user usability evaluation, live browser verification and supervisor approval remain required before

formal submission. The project demonstrates that a transparent, rule-based approach can deliver a functional digital menu and recommendation prototype while making its evidence and limitations visible.

ACKNOWLEDGEMENTS

The candidate may acknowledge the project supervisor, evaluators, test participants and relevant educational resources after confirming their involvement. This section intentionally does not claim meetings or assistance that have not been verified.

TABLE OF CONTENTS

The chapter and appendix page references below correspond to the numbered report body. Update and verify page references after any final edits.

CHAPTER 1: INTRODUCTION 1
CHAPTER 2: LITERATURE REVIEW 3
CHAPTER 3: METHODOLOGY 6
CHAPTER 4: SYSTEM ANALYSIS AND DESIGN 8
CHAPTER 5: SYSTEM DESIGN AND IMPLEMENTATION 11
CHAPTER 6: CONCLUSION 15
REFERENCES 17
APPENDIX A: INSTALLATION 17
APPENDIX B: AUTOMATED TEST RECORD 17
APPENDIX C: USER QUESTIONNAIRE 18
APPENDIX D: REMAINING SUBMISSION EVIDENCE 18

LIST OF FIGURES

Figure 3.1. Recommendation eligibility and scoring logic

Figure 4.1. Use-case model

Figure 4.2. Logical application architecture

Figure 4.3. Entity relationship model

Figure 5.1. Recommendation workflow

LIST OF TABLES

Table 1.1. Project objectives

Table 2.1. Relevant system comparison

Table 3.1. Development steps

Table 4.1. Functional requirements

Table 4.2. Data resources

Table 5.1. Scoring components

Table 5.2. Test evidence

Table 5.3. Verification limitations

LIST OF APPENDICES

Appendix A. Installation and reproducibility guide

Appendix B. Test matrix and actual results

Appendix C. User-evaluation questionnaire (unfilled)

Appendix D. Supervisor and academic integrity evidence to attach

CHAPTER 1: INTRODUCTION

1.1 Background

The digitization of restaurant menus has improved access to prices and dish descriptions, but the amount and quality of accompanying nutritional information vary. Customers interested in comparing meals may need to examine calories, macronutrients and ingredient labels across several menu items. Independent nutrition trackers such as MyFitnessPal and Cronometer demonstrate interest in structured food information; however, they primarily emphasize dietary logging and analysis rather than the specific workflow of browsing fictional restaurant menus and obtaining transparent meal rankings (MyFitnessPal, 2026; Cronometer, n.d.).

World Health Organization (2026) describes adequacy, balance, moderation and diversity as important principles of a healthy diet. An educational menu application may support understanding of nutritional values without claiming to replace professional dietary assessment. SMARTPLATE was specified as a web-based system for digitally displaying restaurant menus, analyzing nutritional values and providing personalized, health-aware recommendations using a rule-based approach. This project converts that description into a testable web application with explicit transparency and safety boundaries.

1.2 Problem Statement

The target problem is fragmented menu information: meal descriptions, serving sizes, nutritional measurements and user-specific selection criteria may be distributed across different sources or absent. If a system displays a recommendation without justification, users cannot easily determine whether it matches a stated calorie target or nutritional priority. A second problem is data uncertainty: an unlisted ingredient or allergen must not be interpreted as a verified absence. A useful prototype therefore needs not only a

menu interface, but also structured nutrition data, reproducible ranking, provenance disclosure and conservative handling of missing information.

1.3 Aim and Objectives

The aim is to design, implement and evaluate a functional web-based prototype that enables users to browse restaurant menus, examine nutrition information and obtain explainable rule-based meal recommendations.

Table 1.1. Project objectives

Objective	Success indicator
O1. Digitize restaurant menus	Restaurant directory, searchable menus and item details are available.
O2. Display nutritional profiles	Six common nutrient fields and serving size are shown, including unknown values.
O3. Personalize meal selection	User preferences lead to deterministic ranked suggestions and exclusion reasons.
O4. Enable menu administration	Authenticated administrator can manage restaurants and meals.
O5. Validate implementation	Automated tests and a documented human-evaluation procedure are provided.

1.4 Project Scope

Included functions are public restaurant browsing, text search, category and vegetarian filtering, calorie/protein/carbohydrate/fat/sodium/fiber displays, preference collection, deterministic scoring, side-by-side meal comparison, browser-stored favorites, administrator authentication and create/read/update/delete operations. The demonstration dataset consists of four fictional restaurants and 48 synthetic meal entries, each labelled illustrative. Medical diagnosis, nutritional disease prediction, automated menu photograph interpretation, live restaurant APIs, payment processing and trained artificial-intelligence models are excluded.

1.5 Target Users

The intended audience includes general customers seeking accessible menu information, health-conscious people tracking broad nutrient preferences, students and working

professionals making quick comparisons, and restaurant personnel who would maintain digital entries. The prototype collects no individual medical diagnoses; calorie targets and other preferences remain in the user's browser.

1.6 Project Timeline

Table 1.2. Planned milestone schedule

Period	Target outcome
15–18 October 2026	Requirements, first full-stack prototype and first documentation.
19–25 October 2026	Core recommendation and administrator functions, early verification.
26 October–1 November 2026	Integration and feature freeze; real-browser inspection.
2–8 November 2026	Real-user evaluation, evidence gathering and completed report review.
9–12 November 2026	Supervisor corrections and final release.
13–15 November 2026	Contingency, printing/signatures and submission.

The implementation was generated as an integrated application on 15 October 2026; the remaining time must be used for review, integration evidence and academic preparation. Actual dates of supervisor consultations and user evaluation must be recorded separately.

CHAPTER 2: LITERATURE REVIEW

2.1 Digital Food and Nutrition Information

Nutrition systems rely on structured descriptions of foods and their nutrients. The United States Department of Agriculture (USDA, n.d.) organizes FoodData Central into multiple data types, including Foundation Foods and Branded Foods. These classifications demonstrate why nutrient values require provenance: an analytical measurement, compiled estimate and branded food label do not represent identical evidence. A system that displays calories without a source or serving size may convey unwarranted certainty.

MyFCD is an additional regional reference to consider for future Malaysian-food data sourcing. SMARTPLATE uses none of these published sources to certify its demonstration dishes. The source categories are included to establish how a real-world dataset could later be gathered and attributed while preserving the integrity of the prototype.

2.2 Review of Existing Systems

MyFitnessPal supports food logging and meal-related nutrition information through its diary workflow. Its public help documentation describes looking up a food, inspecting nutrition and changing serving size (MyFitnessPal, 2026). Cronometer supports detailed food tracking, nutritional breakdowns and micronutrient analysis, emphasizing meal logging and nutritional history (Cronometer, n.d.). SMARTPLATE differs by centering restaurant menu discovery, item comparison, administrator-controlled menus and an explicitly explainable per-meal ranking, rather than serving as a daily intake diary.

Table 2.1. Comparison of related systems

Capability	MyFitnessPal	Cronometer	SMARTPLATE
Primary workflow	Dietary logging	Nutrient diary	Restaurant menu selection
Nutrient detail	Yes	Yes, extensive	Six key measurements
Browse demo restaurants	Not core aim	Not core aim	Yes
Explain heuristic weights	Not assumed here	Not assumed here	Yes (40/30/20/10)
Restaurant admin CRUD	Not core aim	Not core aim	Yes
Source limitations	Varies by data entry	Data emphasis	Illustrative demo explicitly flagged

2.3 Recommendation Approaches

Recommendation systems can use collaborative filtering, content-based methods, learned ranking and rule-based selection. The chosen proposal suggests a rule-based approach; this project therefore uses explicit, auditable eligibility filters followed by numeric scoring. No claim of trained-model intelligence is made. A deterministic

function also simplifies test creation because the same menu and preferences must produce the same ranking.

The present weights emphasize proximity to a user-supplied calorie target (40%), selected nutritional priority (30%), completeness of six nutrient fields (20%) and fiber as a transparent heuristic (10%). These percentages are initial software-design decisions. They are not extracted from clinical research or presumed medically optimal, and the report must not attribute physiological significance to the score.

2.4 Food Safety and Accuracy Limitations

World Health Organization (2026) provides population-level dietary principles, but they cannot establish whether a specific fictional restaurant meal is suitable for an individual. Unknown allergen information is particularly consequential: ingredients may change, cross-contact can occur, and a missing declaration is not assurance. SMARTPLATE treats absent verification as uncertainty and excludes meals when a strict allergen preference cannot be supported. This deliberately conservative behavior may return no results for the demonstration dataset.

2.5 Research Gap and Project Positioning

The identified engineering opportunity is to combine accessible digital menu administration and browsing with an interpretable, rule-based choice workflow in a student-sized web application. Its intended contribution is not novel biomedical evidence or nutritional modeling; it is a demonstrable integration of validated data entry, clear user interactions, transparent scoring and explicit data caveats. Usability outcomes require empirical evaluation and are not inferred from implemented features alone.

CHAPTER 3: METHODOLOGY

3.1 Development Methodology

An iterative software-development process was selected: requirements analysis, data modeling, implementation, verification, review and improvement. This fits a minor project whose deliverable is a working system with associated design and test evidence. The scope is limited to functions approved in the proposal, avoiding model training or live external integrations that would be difficult to substantiate within the project schedule.

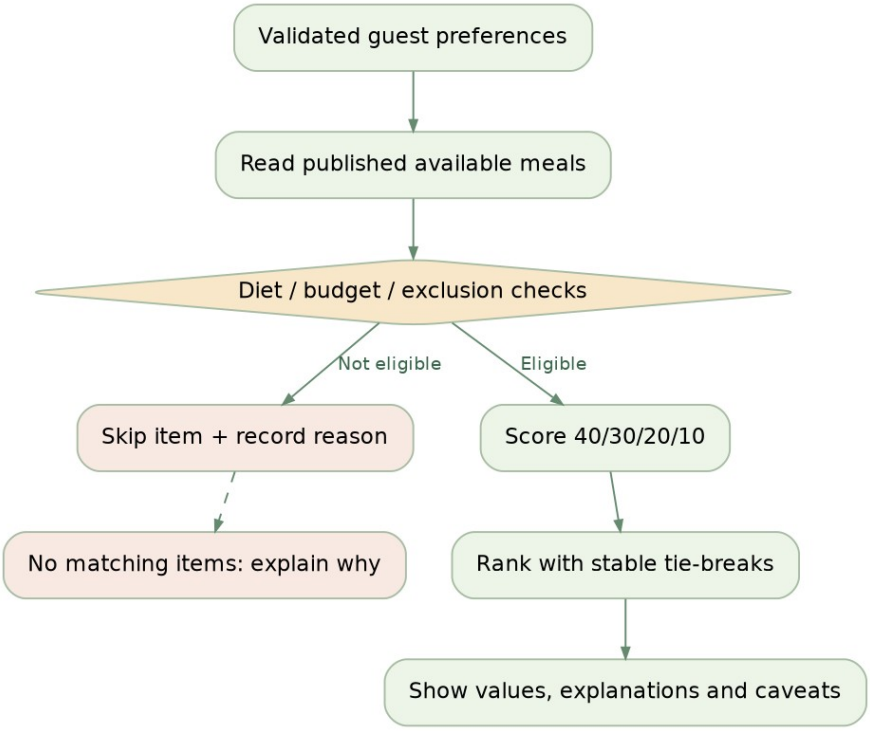


Figure 3.1. Iterative filtering and ranking logic implemented in SMARTPLATE

3.2 Requirements Elicitation

Functional requirements were derived from the proposal's target users and commitments: digital menus, nutritional analysis, individual preferences and a rule-based recommendation system. Technical supporting requirements include maintaining menu data, protecting administrative changes and explaining output to users. The full

traceability list is provided in the software package at docs/REQUIREMENTS.md; the functional subset is reproduced in Chapter 4.

3.3 Design, Development and Implementation

The application is organized into a frontend and a REST API. HTML, CSS and JavaScript render the user experience, while FastAPI handles requests, Pydantic validates input and SQLite persists menu and administrative data. Python functions implement scoring and access restrictions. This architecture was selected instead of a heavier JavaScript stack for self-contained local demonstration and easier reproducibility. The final report documents the actual stack rather than the earlier planning alternatives.

Table 3.1. Development activities and tangible outputs

Activity	Deliverable
Requirements analysis	Functional and nonfunctional specification, acceptance criteria.
Data design	SQLite schema and deterministic illustrative seed.
Web implementation	Responsive browse, detail, preference and comparison pages.
Backend design	Validated REST endpoints and admin authentication.
Recommendation logic	Pure deterministic ranking function with caveats.
Verification	20 passing automated checks; manual and user testing plan.

3.4 System Testing Strategy

Automated unit tests assess numerical ranking, filtering, missing nutrient values, determinism and safety-conservative restrictions. API tests exercise restaurant discovery, menu retrieval, recommendation validation, login, session enforcement, CSRF, administrative CRUD operations and data visibility. Tests run against a temporary SQLite database to avoid altering the demonstration dataset. The test results reported here represent executed automated checks, not browser compatibility or participant satisfaction.

3.5 User Evaluation Method

A separate task-based evaluation is proposed for at least five genuine participants where feasible, representing target customers and student users. They should attempt menu search, nutrition inspection, preference selection, ranking interpretation and meal comparison. The provided questionnaire uses eight five-point rating items supplemented by open comments. The report must be amended to state the real sample size, measures, observed task outcomes and limitations when those sessions occur; no responses are assumed in this draft.

3.6 Ethical Considerations

The prototype does not require users to enter diagnoses or identifiable health records. Browser-local preference storage avoids server-side aggregation of diet preferences, while admin sessions require authentication. Demonstration menu values are explicitly illustrative to reduce risk of misunderstanding. User-testing participants should be informed of the purpose of feedback and any institutional requirements for participation or consent should be followed. Approval and meeting records must reflect actual events.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 Functional Requirements

Table 4.1. Main functional requirements

ID	Requirement	Implementation
FR01–04	Restaurant/menu browse, search and filter	Public endpoints and Explore interface.
FR05–06	Per-serving nutrients and provenance	Meal detail modal and source labels.
FR07–10	Preferences, ranking and no-results explanations	Recommendation engine and Results interface.
FR11–12	Meal comparison and favorites	Browser local storage and comparison table.
FR13–16	Admin authentication, CRUD and audit	Protected admin routes and SQLite records.

4.2 Use Cases

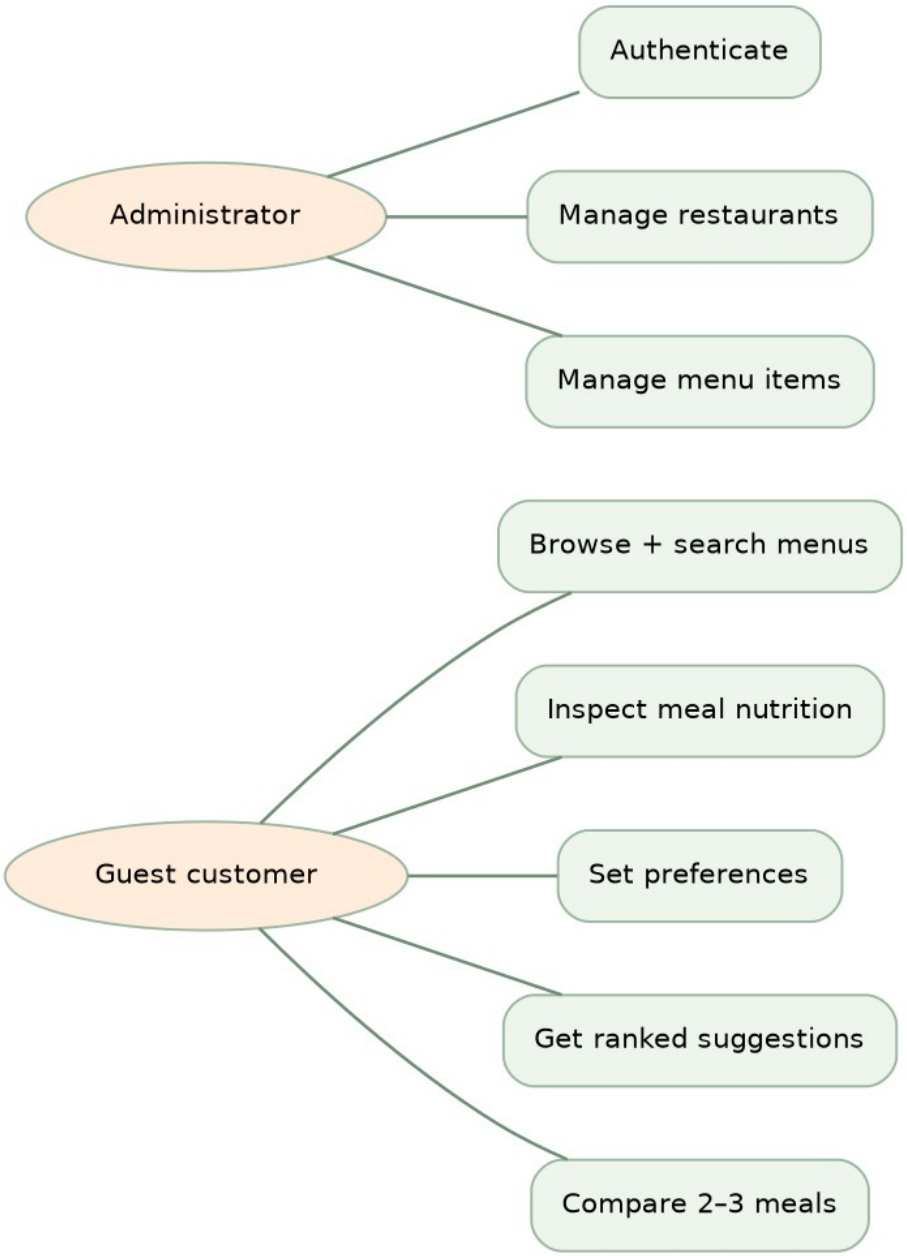


Figure 4.1. Use-case model for guest and administrator actors

Guests access public read-only functions and their personal browser-local preferences. Administrators authenticate before modifying restaurants, menu items or source declarations. Only the admin route has authority to change server-side records. Inability to authenticate must prevent unauthorized CRUD requests even when an endpoint is accessed directly.

4.3 High-Level Architecture

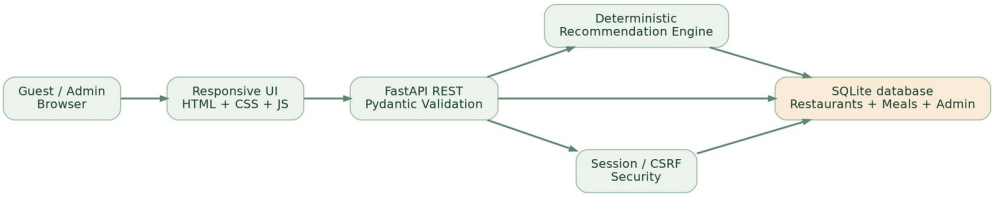


Figure 4.2. Implemented logical application architecture

A single-origin arrangement serves frontend files and API endpoints from FastAPI. A thin client makes JSON requests and renders the corresponding interfaces. Server-side logic handles validation and database access. SQLite stores relational menu records, administrator password hashes, session token hashes and mutation audit records. The recommendation function is separated from HTTP request handling to make its calculation testable and reusable.

4.4 Data Resource Specifications

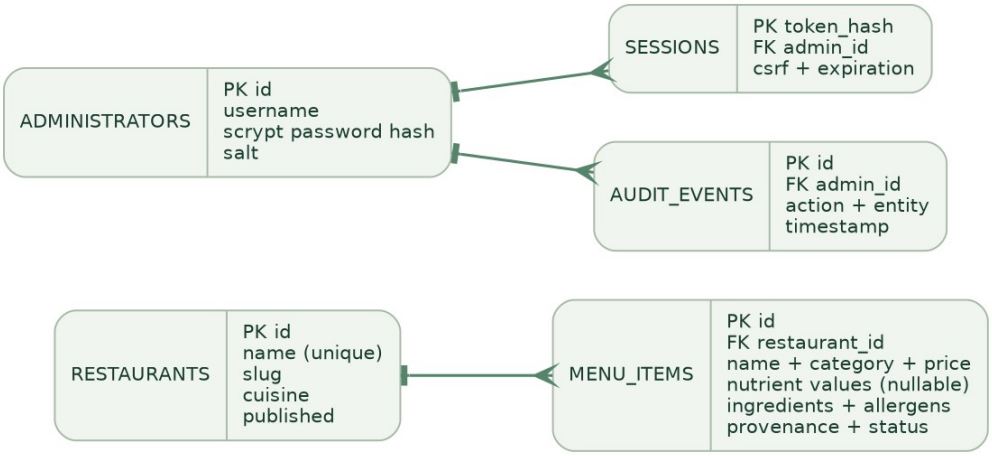


Figure 4.3. Implemented entity relationship model

Table 4.2. Primary data resources

Table	Representative attributes	Purpose
restaurants	id, name, slug, cuisine, published	Venue and public visibility.
menu_items	restaurant_id, nutrient values, ingredients, provenance	Menu content and selection inputs.
administrators	username, salt, scrypt password hash	Admin identification.
sessions	hashed token, admin id, CSRF, expiry	Authenticated administrative sessions.
audit_events	actor, action, entity, timestamp	Change accountability.

4.5 Security and Nonfunctional Design

The system uses Pydantic to constrain request data types, values and lengths. SQL operations use parameterized values rather than building dynamic strings from client input. Admin passwords are derived with scrypt using unique salts, and the server stores only SHA-256 digests of random session tokens. HTTP-only same-site cookies and request CSRF tokens protect administrative edits; cookie Secure should be enabled under HTTPS in a production environment. A basic login throttle limits repeated unsuccessful attempts in the single-process prototype.

The interface includes labeled inputs, keyboard focus indication, reduced-motion consideration and responsive layouts. These are design features; accessibility compliance and cross-device browser compatibility must still be evaluated with actual tools and users. Error states distinguish unavailable network resources, no matching filters and unauthenticated access.

CHAPTER 5: SYSTEM DESIGN AND IMPLEMENTATION

5.1 Implemented User Interface

The homepage introduces the educational system, featured dishes, three steps of use and fictional restaurant cards. Explore provides full-text search, category filters, a restaurant selector and a vegetarian toggle. The meal details dialog shows six nutrients and per-serving size, ingredient and illustrative allergen descriptions, and provenance. The recommendation page requests a target energy value, optional exclusions and a priority before displaying rankings. The comparison page displays selected meals side by side. Administrator pages provide sign-in and record management.

The source repository contains the CSS, HTML and client JavaScript implementing each interface. Real application screenshots should be captured after manual browser inspection and included before final submission; this draft intentionally does not substitute design diagrams for proof of a successful browser walkthrough.

5.2 Database and Data Entry Procedures

Each meal is associated with one restaurant and records name, category, price, serving size, six nullable nutrient measurements, diet status, ingredient and allergen labels, availability and provenance type. Values are validated before INSERT or UPDATE operations. A missing nutrient remains NULL and is displayed as “N/A,” instead of defaulting to zero. Menu editors can mark a meal unavailable and choose how source data were obtained; demonstration entries are not presented as externally verified.

5.3 Recommendation Engine

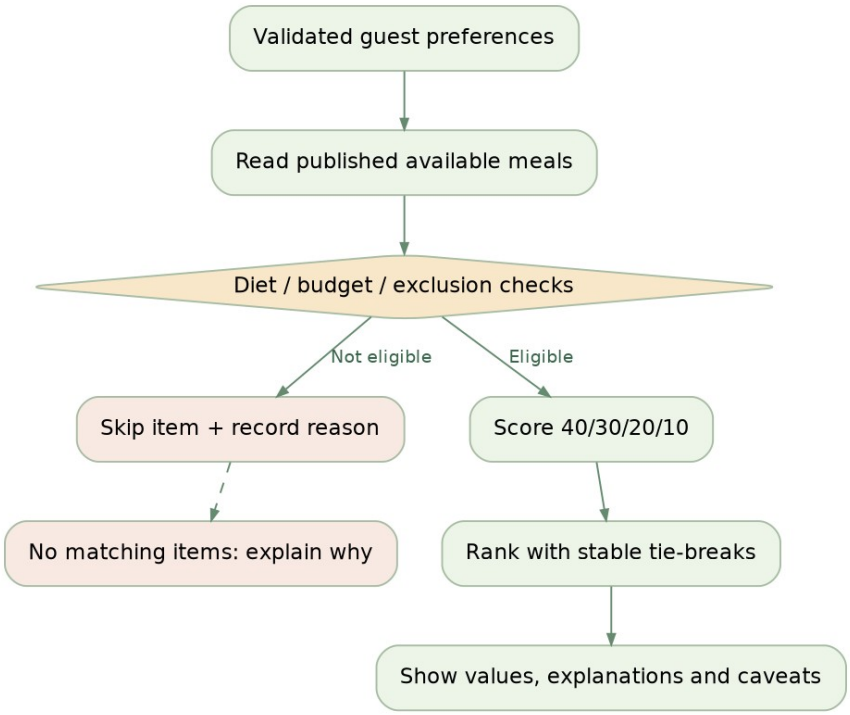


Figure 5.1. Recommendation eligibility, scoring and result rendering

The algorithm first removes unavailable meals, then applies optional restaurant and price restrictions, vegetarian requirements and excluded ingredients or allergens. If a

user specifies strict exclusions and declarations are not verified, the item is excluded instead of being called safe. Items without calories are excluded from numeric ranking. Remaining meals receive a normalized score between zero and 100 computed from four explicitly disclosed parts.

Table 5.1. Recommendation score components

Component	Weight	Calculation / treatment
Calorie closeness	40	40 × max(0, 1 − |actual − target| / max(target, 300)).
Nutrition priority	30	Higher protein, lower sodium, lower carbs or balanced heuristic; missing = 0 points.
Data completeness	20	20 × (known values / 6 displayed nutrients).
Fiber bonus	10	10 × min(1, fiber g / 10); missing = 0 points.

The ranking is sorted in descending score order, with a tie-break based on distance from the target and then stable item ID. Reasons use the actual stored measurements, for example 550 kcal against a 550-kcal target or a measured number of grams of protein. A warning is displayed if nutritional information is unverified or missing. The score provides software-driven matching rather than clinical adequacy.

5.4 Interface Dialogues and User Feedback

Search and filter changes update visible dishes, and descriptive empty states prompt the user to revise filters when there are no matches. Comparison supports at most three entries to keep tables readable on smaller devices. The recommendation page presents a list of exclusions grouped by cause; this helps the user understand why some items were absent without revealing sensitive personal data. Browser local storage retains preferences, favorites and comparisons only on the device used.

5.5 Administrator Module

Admin credentials are created using a command-line setup utility instead of embedding a universal password. After sign-in, the administrator sees restaurant and menu records and can add, update or delete entries using validated forms. CSRF protection and authentication are enforced server-side, so direct calls to write endpoints without a valid session and token are rejected. Each edit produces an audit entry containing the action type, entity reference and time.

5.6 Automated Verification Results

The prepared application was run in the software-creation environment on 15 October 2026. The HTTP health endpoint returned a successful status with 48 records. Twenty tests passed using pytest, covering the scoring algorithm and representative public and private API workflows. The JavaScript file passed Node.js syntax checking. The results represent actual scripted assertions; they do not establish absence of defects or guarantee visual behavior across browsers.

Table 5.2. Executed automated verification

Check group	Examples	Recorded outcome
Ranking unit tests	Target closeness, priority, missing values, tie ordering	Pass
Restriction unit tests	Vegetarian, ingredient, allergen and budget checks	Pass
Public API tests	Restaurants, menu counts, metadata, validation	Pass
Administration API tests	Authentication, CSRF, CRUD, visibility, sign-out	Pass
Security input tests	Invalid URLs and repeated failed login	Pass
Totals	20 tests via pytest	20 passed; 0 failed

5.7 Outstanding Manual and User Evaluation

Table 5.3. Evidence status at report drafting

Evaluation item	Status	Next step
Unit and API automated tests	Completed / passing	Re-run before final submission.

JavaScript syntax	Completed / passing	Re-run if interface code changes.
Live browser desktop and mobile	Not completed	Run on normal Chrome/Edge/Firefox/Safari and capture actual screenshots.
User feedback sessions	Not completed	Run task-based user sessions; summarize real observations.
Supervisor approval	Not recorded	Obtain genuine feedback and signature.

An attempt to conduct Chromium browser walkthroughs in the automated creation environment was blocked by a system administrator URL policy, including localhost URLs. Accordingly, no browser visual test has been marked as passed. The university evaluation should use a regular local installation and collect screenshot and usability evidence directly from that execution.

5.8 Installation and System Maintenance

The repository README contains a cross-platform installation procedure: create a Python virtual environment, install requirements, initialize the SQLite database, create an administrator and run the server. The demo dataset is seeded only if it does not already exist. A maintenance guide describes database backups, development versus HTTPS cookie settings, and the limitations of using SQLite for a production-scale application.

CHAPTER 6: CONCLUSION

6.1 Achievement Against Objectives

The SMARTPLATE prototype implements menu browsing, per-serving nutrition display, guest preferences, a deterministic explanation-producing recommendation algorithm and administrator data management. Its unit and API tests demonstrate selected functional behaviors and error boundaries. The system therefore satisfies the planned software construction scope of the approved proposal as a local educational application, but evidence of user satisfaction and final presentation acceptance remains outstanding until genuine evaluation is carried out.

6.2 Strengths

The main technical strengths are its reproducible local setup, clear API structure, separate unit-testable scoring logic, conservative handling of uncertain exclusions, explicit data provenance and admin-only mutation boundary. Users can inspect the factors behind scores and compare numeric meal attributes rather than being given opaque advice. The software architecture is sufficiently compact to be understood and defended in a minor-project assessment.

6.3 Weaknesses and Limitations

The demonstration menu records and nutrition values are fictional and unverified. No live restaurant feed, laboratory measurement or medical interpretation is included. Ingredient lists may not reflect kitchen preparation or cross-contact, so recommendations cannot certify food safety. Calorie and nutrient targets are not tailored clinically, and score weights are chosen engineering heuristics. Guest data are browser-local, the server is a single-instance SQLite application, and formal browser/usability evaluation remains to be performed.

6.4 Recommended Future Work

Future improvements should prioritize reliable restaurant-supplied nutrition data with source documents, standardized portion estimation, independent nutritional review and auditable allergen handling. Other improvements include multilingual support, deployed PostgreSQL storage, progressive accessibility auditing, real restaurant workflows and rigorous participant evaluation. Any machine-learning extension should require an explicit research question, verified data and comparative metrics rather than adding an opaque model simply to justify the term “intelligent.”

6.5 Final Statement

SMARTPLATE provides a practical demonstration of how digital menu management and explainable rule-based selection can be combined in a coherent web system. The highest priority before formal submission is to replace outstanding evaluation placeholders with actual evidence, verify the report with the candidate and supervisor, and preserve the clear distinction between illustrative demonstration data and reliable dietary information.

REFERENCES

Cronometer. (n.d.). Track food & calories. https://mobile.cronometer.com/features/track-food.html

MyFitnessPal. (2026, September 23). How to log food to your diary. https://support.myfitnesspal.com/hc/en-us/articles/360032274592-How-to-log-food-to-your-diary

United States Department of Agriculture, Agricultural Research Service. (n.d.). FoodData Central: Data documentation. https://fdc.nal.usda.gov/data-documentation/

World Health Organization. (2026, January 26). Healthy diet. https://www.who.int/news-room/fact-sheets/detail/healthy-diet

Malaysian Food Composition Database. (n.d.). Malaysian Food Composition Database. https://myfcd.moh.gov.my/

APPENDIX A: INSTALLATION

Python 3.11 or newer is required. The submitted source directory includes requirements.txt, run.py and the smartplate package. Install packages in a virtual environment, run “python -m smartplate.cli init”, then “python run.py”. Open http://127.0.0.1:8000. Create a unique admin account via “python -m smartplate.cli create-admin --username admin” and never include its password in the submitted report.

APPENDIX B: AUTOMATED TEST RECORD

Run “python -m pytest -q”. The reported baseline as of 15 October 2026 is 20 passed. Detailed assertions reside in tests/test_api.py and tests/test_recommender.py, and

docs/TEST_RESULTS.md. Record a dated fresh screenshot or console transcript at final verification.

APPENDIX C: USER QUESTIONNAIRE (UNFILLED)

Participant code: _____ Device type: _____ Date: _____

For each statement choose 1 (strongly disagree), 2, 3, 4 or 5 (strongly agree).

1. Finding a restaurant and dish was easy. Rating: ____

2. Nutrition labels and units were clear. Rating: ____

3. Setting preferences was straightforward. Rating: ____

4. The explanation of meal ranking was understandable. Rating: ____

5. I understood that the demo nutrients were illustrative. Rating: ____

6. I understood the limitations of allergy information. Rating: ____

7. The interface was usable on my device. Rating: ____

8. Overall, comparison of meals became easier. Rating: ____

Comments and suggested improvements:

_____

APPENDIX D: REMAINING SUBMISSION EVIDENCE

Attach authentic real-browser screenshots, observed user-testing table and questionnaire analysis, supervisor feedback log and required signed approval/declaration pages. These documents have not been simulated in this draft.